

CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 18: Pipelined Datapath I

Instructor: Justin Yokota

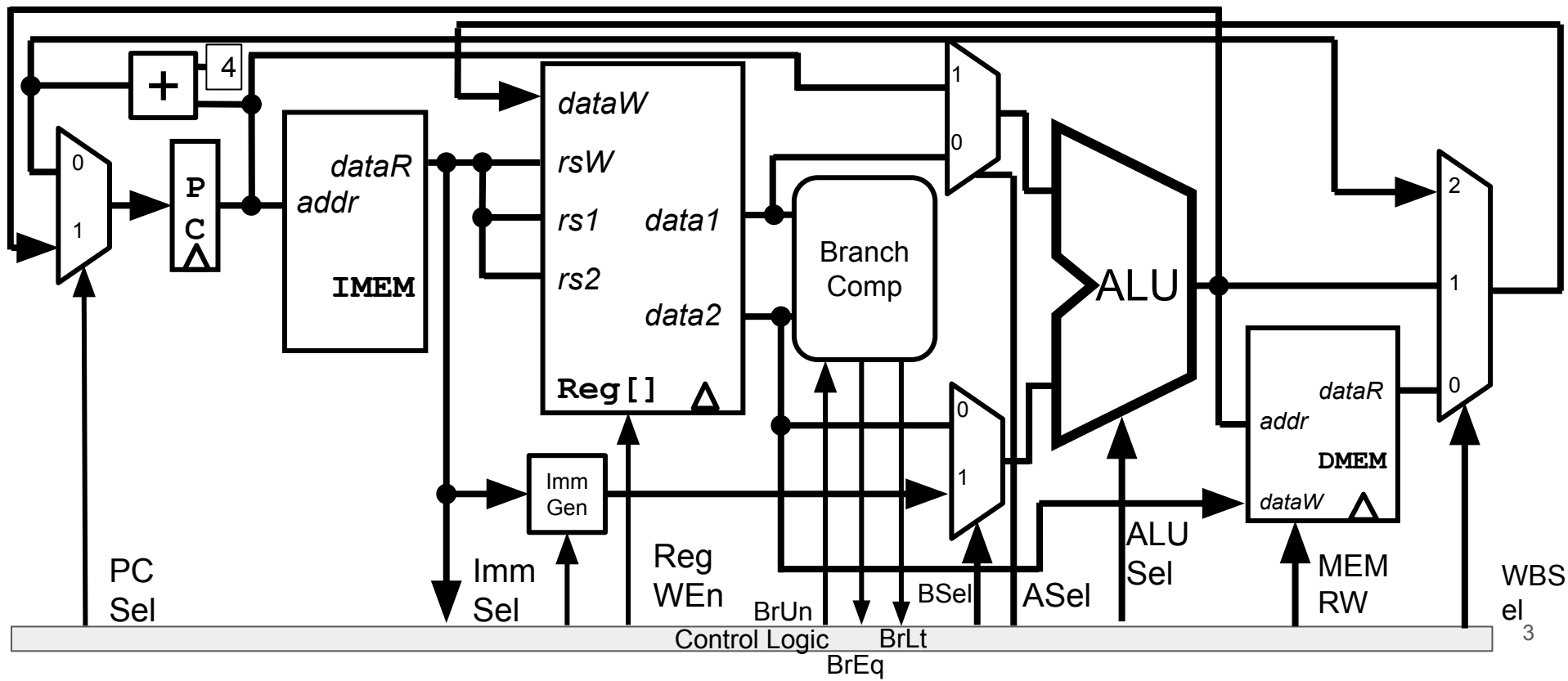
Agenda

- Instruction Timing
- Performance Measures
- "Iron Law" of Processor Performance
- Intro to Pipelining: Laundry
- Aside: Energy Efficiency
- RISC-V: Improving Throughput
- RISC-V Pipelined Datapath

Complete RISC-V Datapath!

CS 61C

Summer 2026



Review

- We have built a processor!
 - Capable of executing all RISC-V instructions in one cycle each
- Not all HW units used by all instructions
 - The **critical path changes** based on instruction.
- 5 Phases of execution
 - IF, ID, EX, MEM, WB
 - Not all instructions are active in all phases
- Controller specifies how to execute instructions
 - Implemented as ROM or logic ([see last lecture](#))

How to make better?

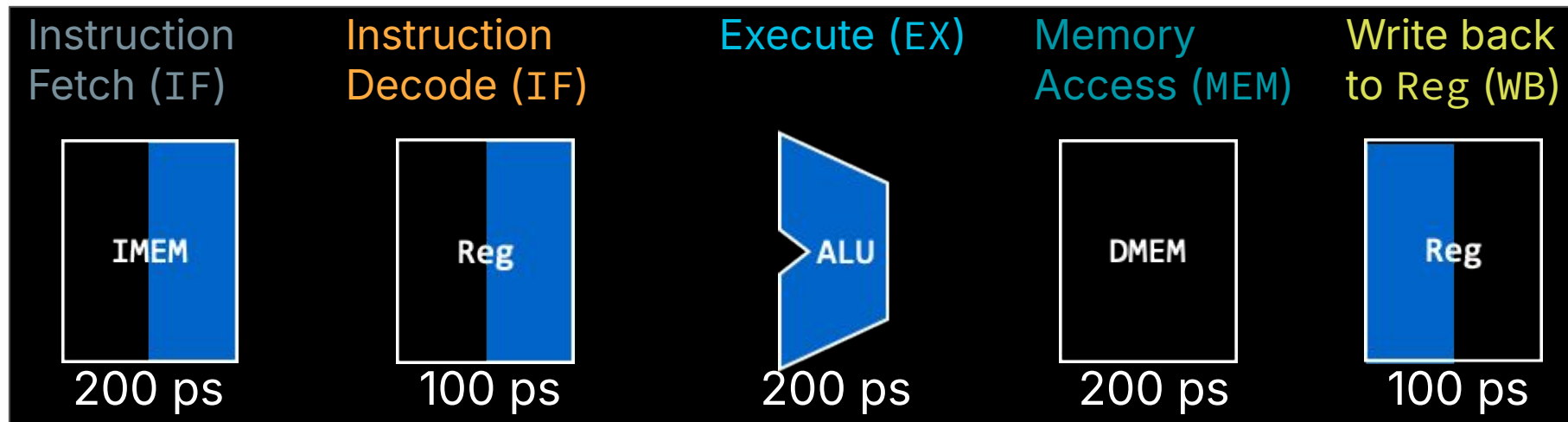
- Our single-stage datapath is reasonably fast, but it's not as efficient as it could be.
 - Most components do nothing most of the time; IMEM's idle while the ALU's running, for example
- Goal: Design a more efficient *pipelined* CPU.
 - How? Start running the next instruction *while the current instruction is still running*
- Today: Develop a framework for analyzing CPU performance
- Tomorrow: Apply those principles to our RISC-V CPU

Agenda

- **Instruction Timing**
- Performance Measures
- "Iron Law" of Processor Performance
- Intro to Pipelining: Laundry
- Aside: Energy Efficiency
- RISC-V: Improving Throughput
- RISC-V Pipelined Datapath

Instruction Timing, Divided by 5 Phases

- To estimate maximum clock frequency, analyze at a higher level.
- Assume each phase is **dominated** by major functional HW units:



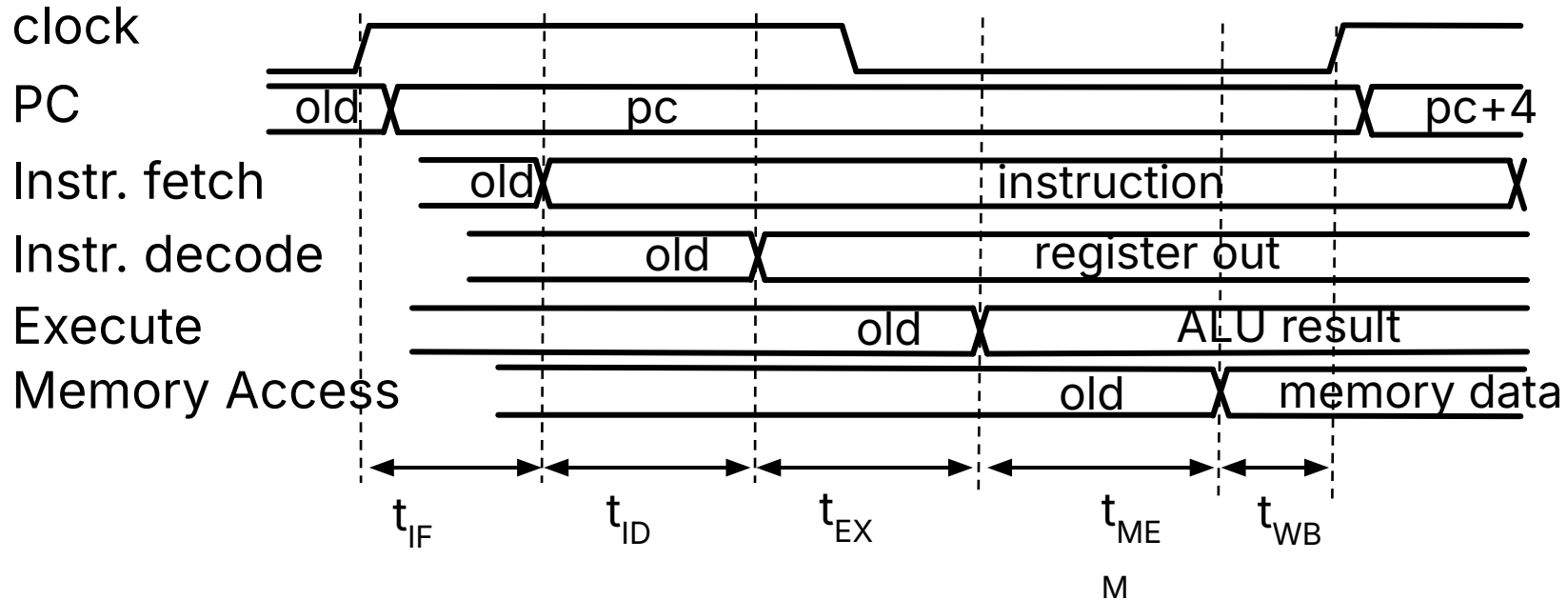
Compute Clock Period

- The clock cycle must encompass the **longest critical path delay** incurred by any instruction.

Instruction	IF (200ps)	ID (100ps)	EX (200ps)	MEM (200ps)	WB (100ps)	Total
add	X	X	X		X	600ps
beq	X	X	X			500ps
jal	X	X	X			500ps
<u>lw</u>	X	X	X	X	X	800ps
sw	X	X	X	X		700ps

- Maximum Clock Frequency:
 - $f_{\max} = 1/800 \text{ ps} = 1.25 \text{ GHz}$
 - However, not all instructions use all HW blocks.
- Many blocks are idle most of the time...!
 - Conversely, each phase just takes 200 ps $\rightarrow$ could clock 5GHz...??

5-Phase Timing, In Detail



Agenda

- Instruction Timing
- **Performance Measures**
- "Iron Law" of Processor Performance
- Intro to Pipelining: Laundry
- Aside: Energy Efficiency
- RISC-V: Improving Throughput
- RISC-V Pipelined Datapath

Performance Measurement & Improvement

- “Our” RISC-V Single-Cycle Datapath executes instructions at 1.25GHz.
 - 1 instruction every 800 ps
 - Frequency unit $\text{Hz} = \text{s}^{-1}$
- Can we improve our processor’s performance? How do we make our processor better?
- Not so obvious what we mean... What does "better" mean?
 - Instruction execution time?
 - Total # instructions executed per unit time?
 - Energy per instruction?
 - Cost over lifetime of chip?
- Generally in engineering, optimizing one aspect pessimizes other aspects

Transportation Analogy

	Sports Car 	Bus 
Passenger Capacity	2 people	50 people
Travel Speed	200 mph	50 mph
Gas Mileage	5 mpg	2 mpg

Which car is better?

Transportation Analogy

CS 61C

Summer 2026

	Sports Car 	Bus 
Passenger Capacity	2 people	50 people
Travel Speed	200 mph	50 mph
Gas Mileage	5 mpg	2 mpg

Which car is better?

- Depends on what we mean by better.
 - Better to run a race, or better to transport people in a city?

Transportation Analogy

	Sports Car 	Bus 
Passenger Capacity	2 people	50 people
Travel Speed	200 mph	50 mph
Gas Mileage	5 mpg	2 mpg

Which car is better at transporting 100 people 50 miles?

- Still depends on what we mean by better!

Transportation Analogy

CS 61C

Summer 2026

	Sports Car 	Bus 
Passenger Capacity	2 people	50 people
Travel Speed	200 mph	50 mph
Gas Mileage	5 mpg	2 mpg

Which car is better at transporting 100 people 50 miles?

Trip time per ride	15 minutes ✓	60 min
Time for 100 passengers	750 min (50 2-person trips)	120 min ✓ (2 50-person trips)
Gallons per passenger	5 gallons	0.5 gallons ✓

Measuring Performance in Computers

- We'll look at 3 different measures of computer performance:

Transportation Analogy	Measure	In a Computer
Trip time per ride	Latency	Instruction latency
Time for 100 passengers	Latency	Program execution time (e.g., time to update display)
Passengers/hour	Throughput	Total tasks per unit time (e.g., # of server requests handled per hour)
Gallons per passenger	Energy Efficiency	Energy per task (e.g., how many movies you can watch per battery charge)

Agenda

- Instruction Timing
- Performance Measures
- **"Iron Law" of Processor Performance**
- Intro to Pipelining: Laundry
- Aside: Energy Efficiency
- RISC-V: Improving Throughput
- RISC-V Pipelined Datapath

“Iron Law” of Processor Performance

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Cycles}}{\text{Instruction}} \frac{\text{Time}}{\text{Cycle}}$$

- The "Iron Law" of Processor Performance, which states how much time it takes to run a program.
 - Implicitly defines "performance" as program execution time
- In order to speed up our code, we need to improve one of:
- Instructions/Program
- Cycles/Instruction
- Time/Cycle

“Iron Law” of Processor Performance

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Cycles}}{\text{Instruction}} \frac{\text{Time}}{\text{Cycle}}$$

Instructions/Program

Determined by:

- Task specification (what we want to do)
- Algorithm ($O(N^2)$ vs $O(N)$)
- Programming Language
- Compiler
- ISA

“Iron Law” of Processor Performance

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Cycles}}{\text{Instruction}} \frac{\text{Time}}{\text{Cycle}}$$

Cycles/Instruction

Determined by:

- ISA
- Processor Implementation

“Iron Law” of Processor Performance

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Cycles}}{\text{Instruction}} \frac{\text{Time}}{\text{Cycle}}$$

Time/Cycle

Determined by:

- Processor Implementation (Critical Path)
- Technology
 - e.g., 5nm vs. 14nm
- Supply voltage (e.g., lower voltage reduces transistor speed but improves energy efficiency)

“Iron Law” of Processor Performance

$$\underbrace{\frac{\text{Time}}{\text{Program}}}_{\text{can measure}} = \underbrace{\frac{\text{Instructions}}{\text{Program}}}_{\text{can count}} \underbrace{\frac{\text{Cycles}}{\text{Instruction}}}_{\text{CPI (average)}} \underbrace{\frac{\text{Time}}{\text{Cycle}}}_{1/(\text{clock rate})}$$

On the same program benchmark, processors will have different **CPI** because of differences in:

- ISA (CISC vs RISC)
- Processor implementation
 - CS61C single cycle: CPI = 1
 - **Pipelined** processor: CPI $\approx$ 1
 - Complex instructions (CPI $\gg$ 1)
 - Superscalar: CPI $<$ 1

Which Processor is Faster?

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Cycles}}{\text{Instruction}} \frac{\text{Time}}{\text{Cycle}}$$

For some task (e.g., image compression), suppose:

	Processor A	Processor B
# instructions	1 million	1.5 million
Average CPI	2.5	1
Clock rate, f	2.5 GHz	2 GHz

Which processor, A, or B, will execute this program faster?

Which processor will execute this program faster?

CS 61C

Summer 2026

A (1M instructions, 2.5 CPI, 2.5 GHz)

B (1.5M instructions, 1 CPI, 2 GHz)

The same

Don't Know



Which processor will execute this program faster?

CS 61C

Summer 2026

0%

B (1.5M instructions, 1 CPI, 2 GHz)

0%

The same

0%

Don't Know

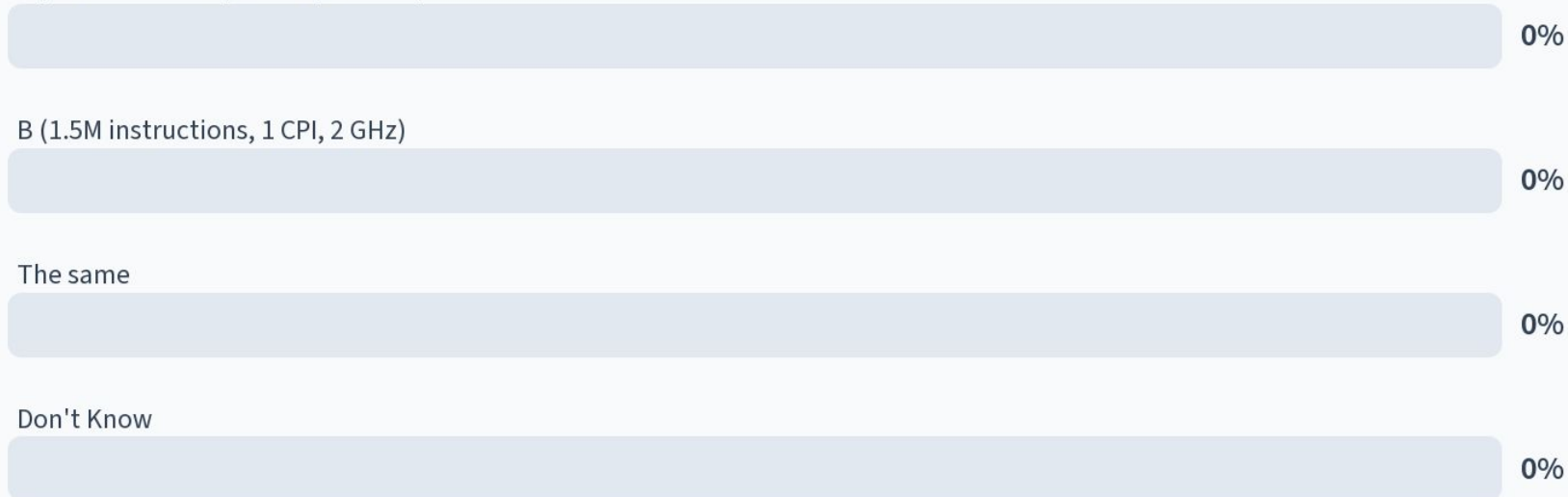
0%



Which processor will execute this program faster?

CS 61C

Summer 2026



Which Processor is Faster? Processor B

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Cycles}}{\text{Instruction}} \frac{\text{Time}}{\text{Cycle}}$$

For some task (e.g., image compression), suppose:

	Processor A	Processor B
# instructions	1 million	1.5 million
Average CPI	2.5	1
Clock rate, f	2.5 GHz	2 GHz
Program execution time	1 ms	0.75 ms 

Despite executing more instructions and having a slower clock rate, Processor B is faster for this task!

Which Processor is Faster? Processor B

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Cycles}}{\text{Instruction}} \frac{\text{Time}}{\text{Cycle}}$$

For some task (e.g., image compression), suppose:

	Processor A	Processor B
# instructions	1 million	1.5 million
Average CPI	2.5	1
Clock rate, f	2.5 GHz	2 GHz
Program execution time	1 ms	0.75 ms ✓
Throughput	1 inst/ns	2 inst/ns ✓

What's the difference between CISC and RISC?

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Cycles}}{\text{Instruction}} \frac{\text{Time}}{\text{Cycle}}$$

What advantages do CISC and RISC systems have with respect to processor performance?

Recall:

CISC philosophy: Have an instruction for as many things as you can fit on the chip

RISC philosophy: Have as few instructions as possible to perform desired work

What's the difference between CISC and RISC?

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Cycles}}{\text{Instruction}} \frac{\text{Time}}{\text{Cycle}}$$

Who wins in Instructions/Program?

- CISC: More complicated instructions = fewer instructions same program

Who wins in Cycles/Instruction?

- Uncertain: Depends on exact processor (and we currently do 1 instruction/cycle always)

Who wins in Time/Cycle?

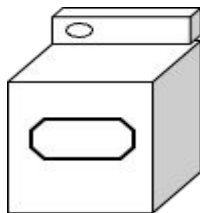
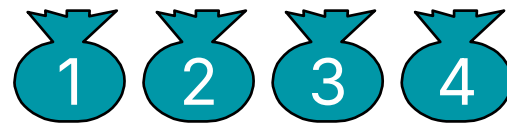
- RISC: Smaller instruction set = less chance for a "bad" critical path

Agenda

- Instruction Timing
- Performance Measures
- "Iron Law" of Processor Performance
- **Intro to Pipelining: Laundry**
- Aside: Energy Efficiency
- RISC-V: Improving Throughput
- RISC-V Pipelined Datapath

Pipelining Analogy: Laundry

- Four customers (1, 2, 3, 4) each have one load of clothes to wash, dry, fold, and put away.



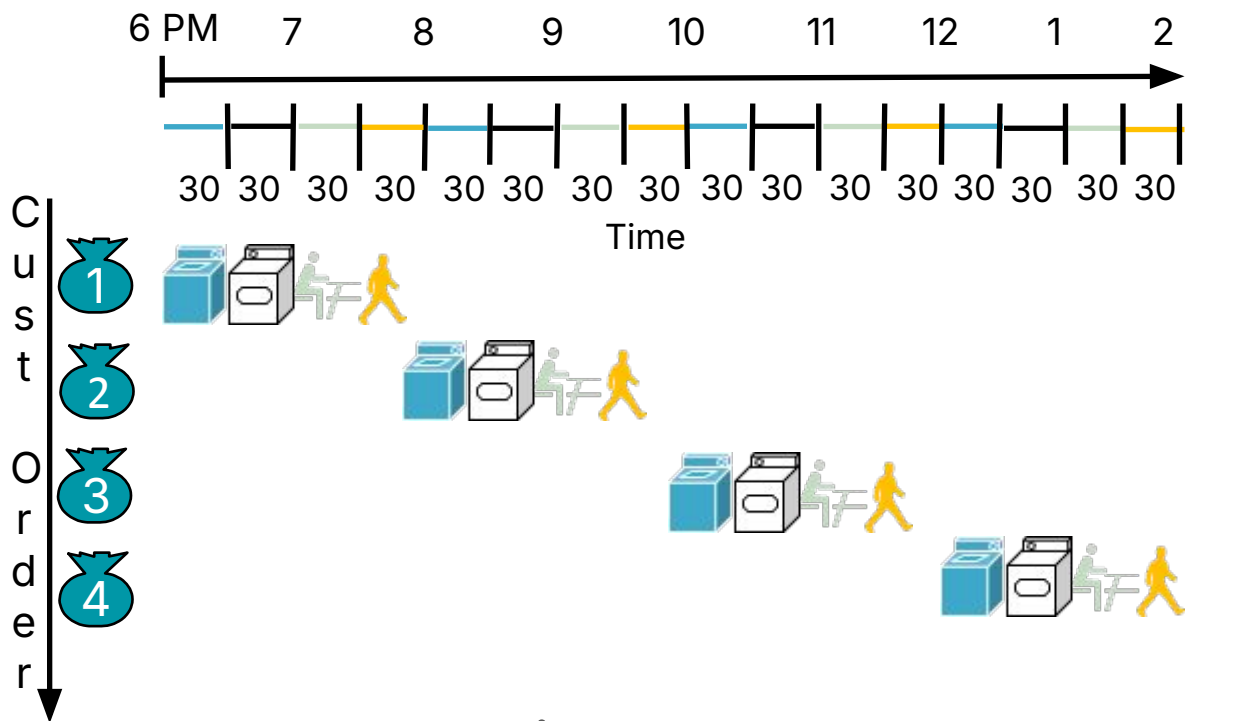
Washer Stage
30 minutes

Dryer Stage
30 minutes

“Folder” Stage
30 minutes

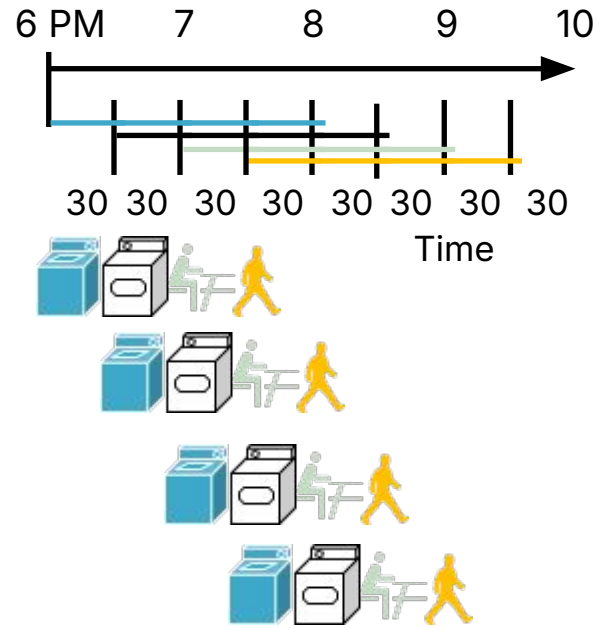
“Stasher” Stage
30 minutes
(put clothes in
drawers)

Sequential Laundry



Sequential laundry takes
8 hours for 4 loads!

Parallel Laundry



Parallel laundry takes
3.5 hours for 4 loads!

Latency vs Throughput

Each load of laundry takes the same amount of time (or potentially longer if the stages aren't the same length)

But because more parts are being used at once, we get more loads done per hour

		Sequential	Pipelined
Program execution time	Time to finish all 4 loads	8 hours	3.5 hours
Instruction Latency	Time to finish a single load	2 hours	2 hours
Instruction Throughput	Avg number of loads per 30 min	0.25	1

Pipelining increases instruction throughput, **not** instruction latency.

A note on CPI

With a pipeline, each load of laundry takes 4 cycles (of 30 minutes each) to complete, but we complete one load each cycle.

Is it more useful to treat CPI as 4, or 1?

		Sequential	Pipelined
Program execution time	Time to finish all 4 loads	8 hours	3.5 hours
Instruction Latency	Time to finish a single load	2 hours	2 hours
Instruction Throughput	Avg number of loads per 30 min	0.25	1

A note on CPI

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Cycles}}{\text{Instruction}} \frac{\text{Time}}{\text{Cycle}}$$

If we use CPI of 4 (latency of one instruction), then that cancels out the 4x speedup of Time/Cycle

- Where did our performance improvement go?
- Answer: Multiple instruction-cycles are being run per clock cycle

If we treat CPI as 1 (Inverse of Throughput), we get the steady-state number of instructions that get run, and see a 4x speedup.

- We'll use CPI to mean this from now on.

Agenda

- Instruction Timing
- Performance Measures
- "Iron Law" of Processor Performance
- Intro to Pipelining: Laundry
- **Aside: Energy Efficiency**
- RISC-V: Improving Throughput
- RISC-V Pipelined Datapath

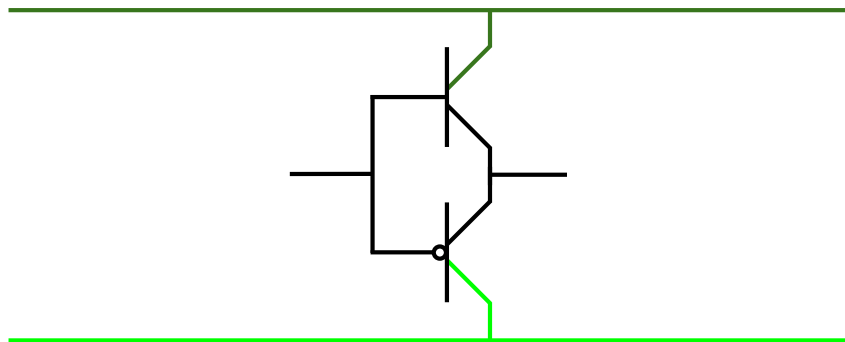
Max Energy Efficiency

$$\frac{\text{Energy}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Energy}}{\text{Instruction}}$$

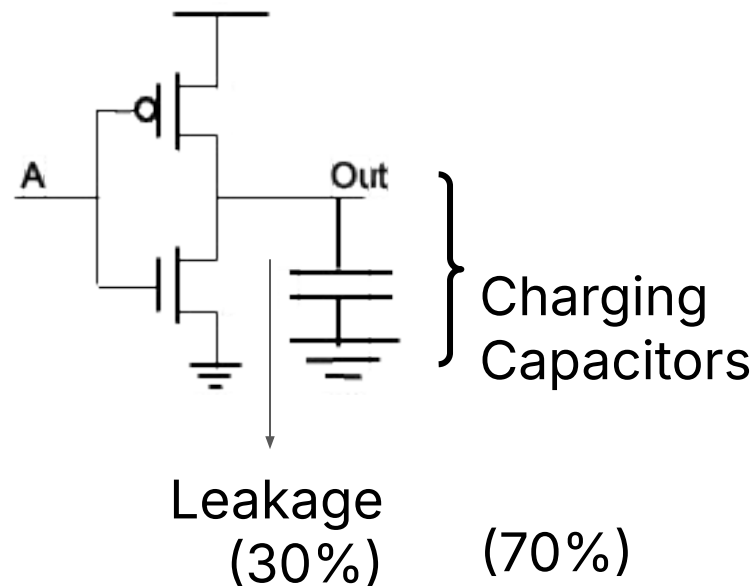
Energy cost is another factor we want to minimize

Optimizing the energy it takes to execute a program means **minimizing** energy consumed per program.

Energy Consumption of Transistors



Schematic



Energy Efficiency

$$\frac{\text{Energy}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \frac{\text{Energy}}{\text{Instruction}}$$

Energy cost of a capacitor is proportional to CV^2

- C = capacitance
 - Depends on hardware technology, and processor features, e.g., # of cores
- V = supply voltage, e.g., 1V

To improve energy efficiency per task, reduce **capacitance and voltage**.

Power-Constrained Systems

$$\text{Power} = \frac{\text{Energy}}{\text{Second}} = \frac{\text{Energy}}{\text{Instruction}} \frac{\text{Instructions}}{\text{Second}}$$

Energy efficiency is a key metric in all computing devices.

This equation introduces an energy constraint:

- If system is **constrained by power**, we need **better energy efficiency** to get more performance at the same power.
- For energy-constrained systems, we need better energy efficiency to prolong battery life.

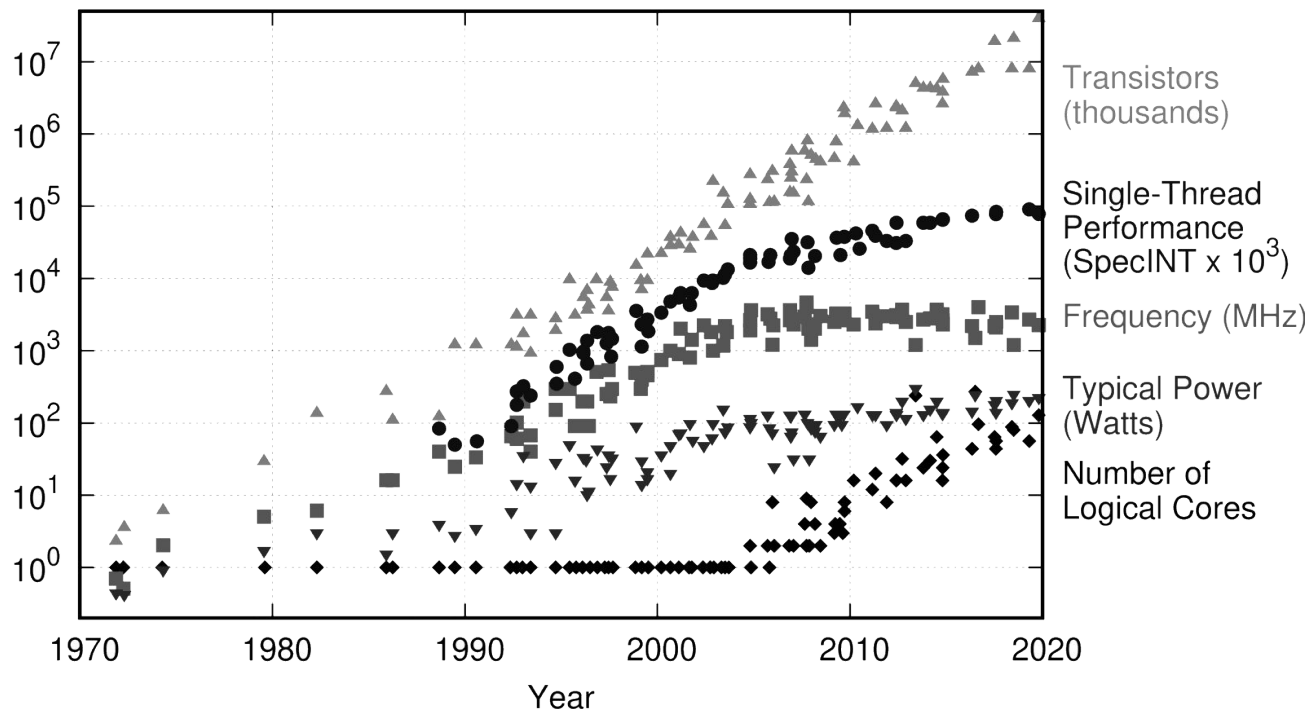
If power increases, heat dissipation becomes a problem - need more power for cooling

Dennard Scaling

- **Dennard scaling:**
 - In 1974, Robert Dennard observed that power density remained constant for a given area of silicon
 - **Reducing dimensions of transistor**
 - reduces capacitance + voltage
 - **reduces power** (proportional to CV^2) proportionally
- **Combined with Moore's Law:**
 - Moore: (# of transistors in chip can be doubled every 2 years)
 - Dennard: Can be accomplished without changing power consumption/area
 - → **Performance per joule doubles** every 1.5 years.

Processor Performance: Trends

48 Years of Microprocessor Trend Data



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2019 by K. Rupp

Power constrains Clock Speed

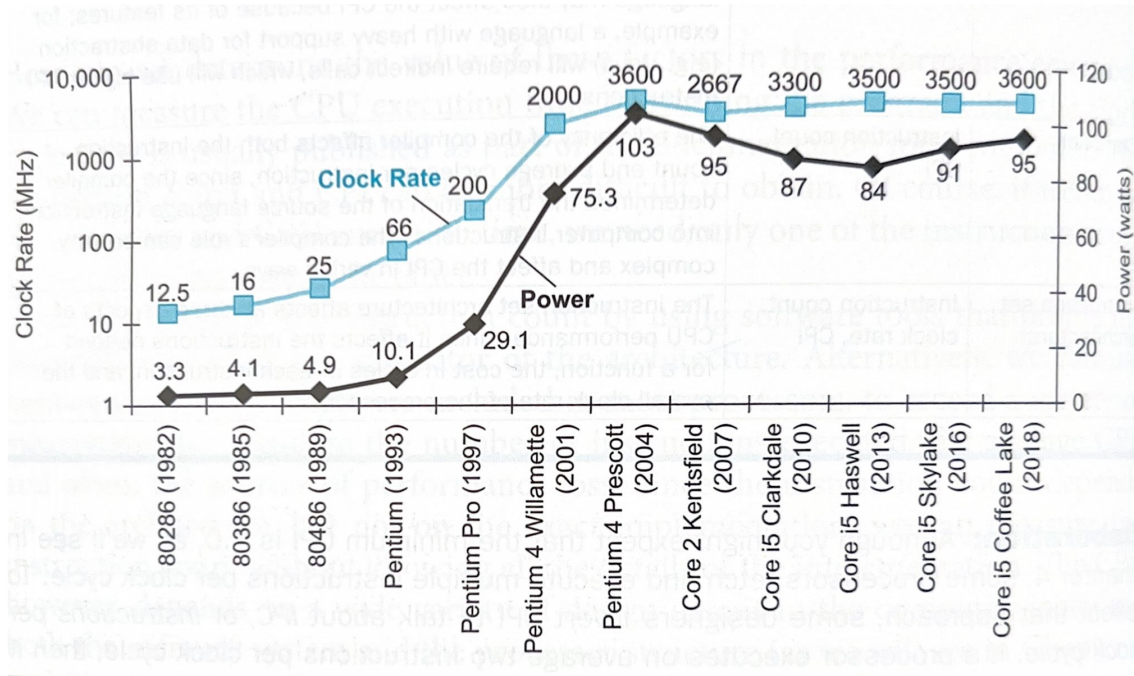


FIGURE 1.16 Clock rate and power for Intel x86 microprocessors over nine generations and 36 years. The Pentium 4 made a dramatic jump in clock rate and power but less so in performance. The Prescott thermal problems led to the abandonment of the Pentium 4 line. The Core 2 line reverts to a simpler pipeline with lower clock rates and multiple processors per chip. The Core i5 pipelines follow in its footsteps.

The End of Dennard Scaling?

- In recent years, industry has not reduced supply voltage much.
 - Reducing supply voltage further would mean increasing “**power leakage**,” where transistor switches don’t fully turn off (more like dimmer switch than on-off switch)
- This “**power wall**” becomes a growing concern!
- Also, size of transistors and hence capacitance, not shrinking as much as before between transistor generations
 - Need to go to 3D
- Combined with the end of Moore's Law: we can no longer rely on steady technological improvement to make our code faster for us
 - Need to distribute our workload to multiple CPUs (See Parallelism section)

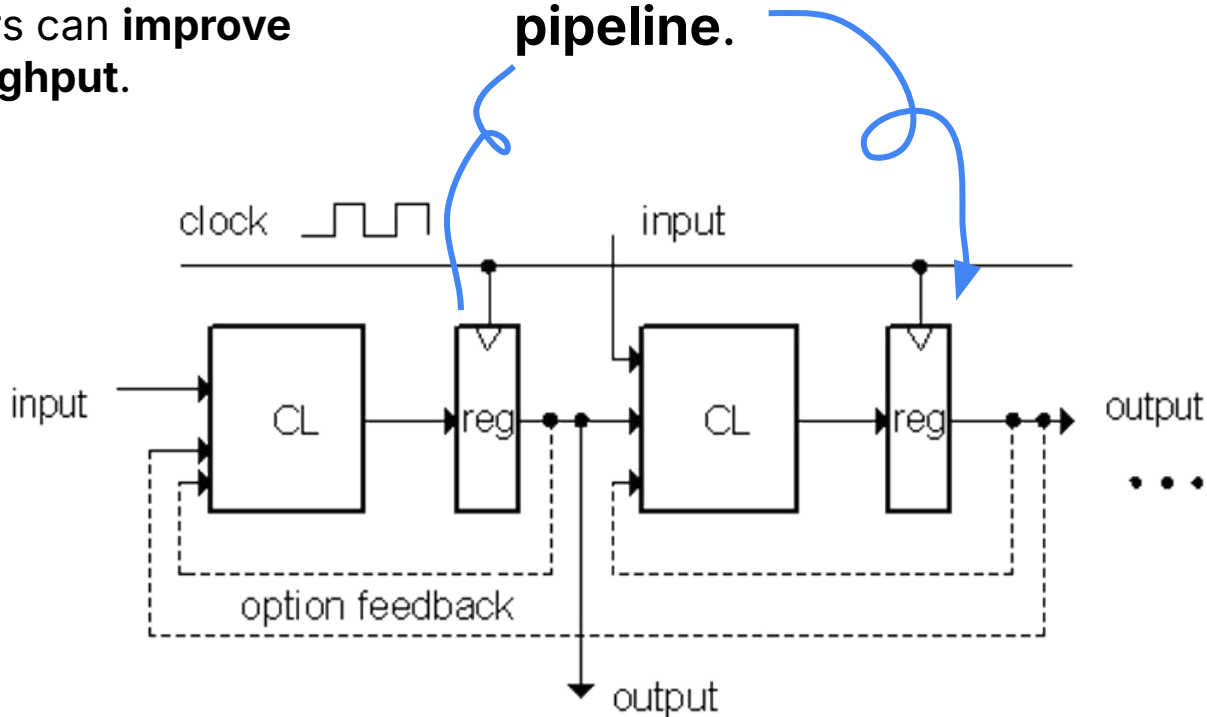
Agenda

- Instruction Timing
- Performance Measures
- "Iron Law" of Processor Performance
- Intro to Pipelining: Laundry
- Aside: Energy Efficiency
- **RISC-V: Improving Throughput**
- RISC-V Pipelined Datapath

General Model for Synchronous Systems, again

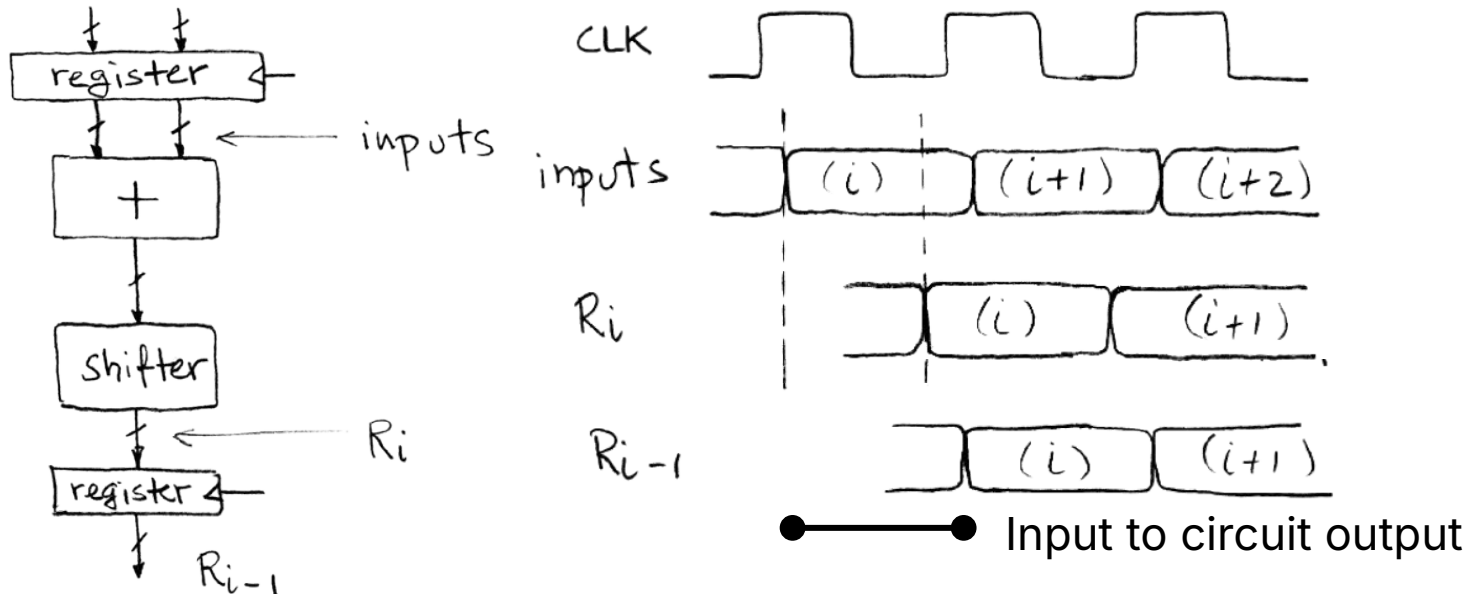
The idea: registers can **improve instruction throughput**.

This creates a **pipeline**.



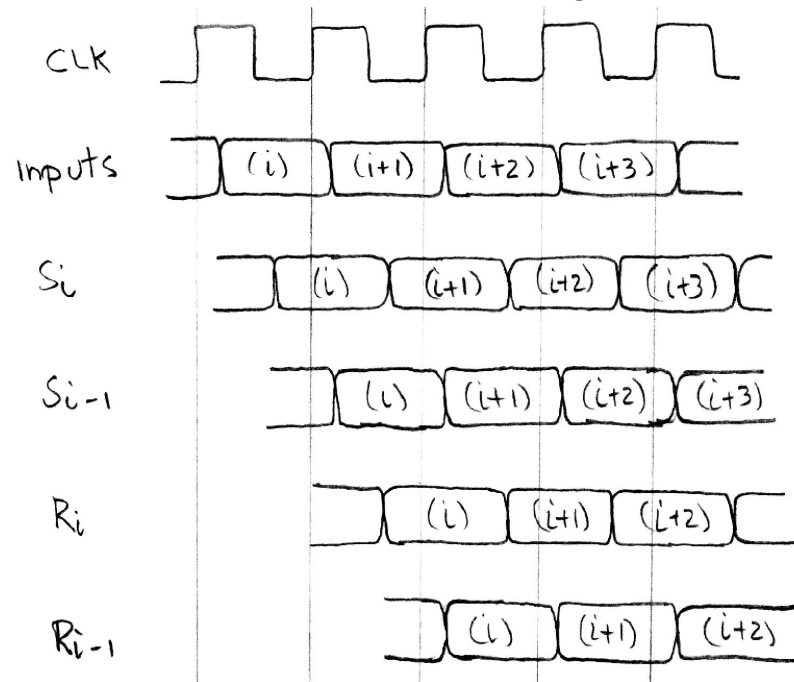
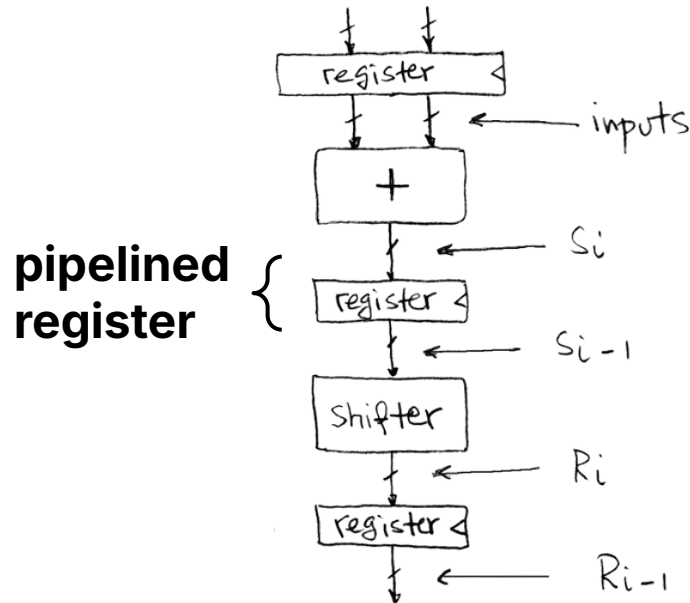
Non-Pipelined Adder/Shifter Circuit

- **Without a pipeline**, delay of 1 clock cycle to compute output.
 - Clock period is limited by the propagation delay of adder + shifter.



Pipelined Adder/Shifter Circuit

- **Extra Registers** are often added to help increase the clock freq.
- Pipeline: Shifter outputs **while the next** adder input is being computed

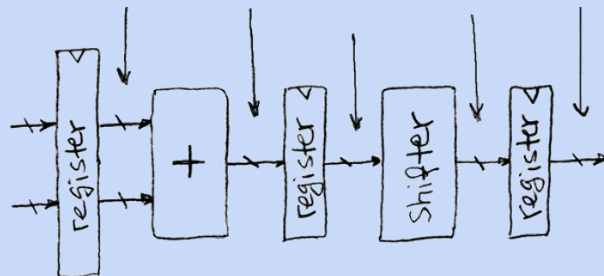
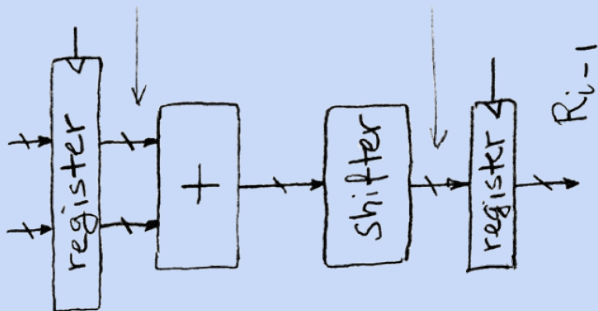


Latency, Throughput Practice

Let's say we have the following properties of our components:

- Register clk-to-q: 30 ps
- Register setup: 20 ps
- Adder delay: 150 ps
- Shifter delay: 50 ps

What is the latency and throughput of the single-cycle and pipelined circuits?



Single-Cycle Latency, Throughput

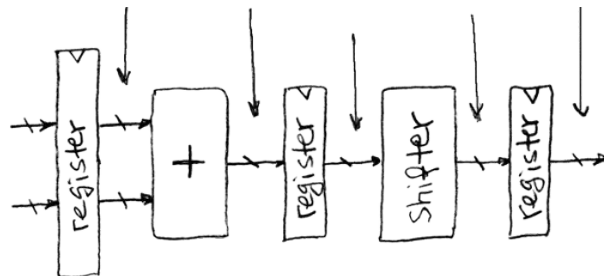
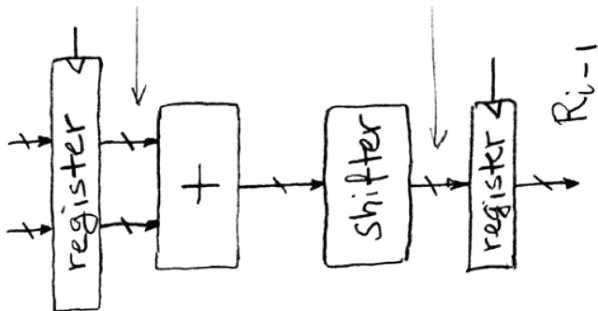
Critical path delay = $t_{clk_q} + t_{Add}$
+ $t_{Shift} + t_{Setup}$

= $30 + 150 + 50 + 20$ ps = 250 ps

Latency = 1 cycle = 250 ps

Throughput = $1/\text{cycle}$ = 4 GHz

- Register clk-to-q: 30 ps
- Register setup: 20 ps
- Adder delay: 150 ps
- Shifter delay: 50 ps



Pipeline Latency, Throughput

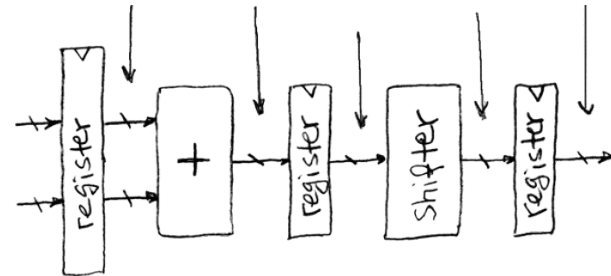
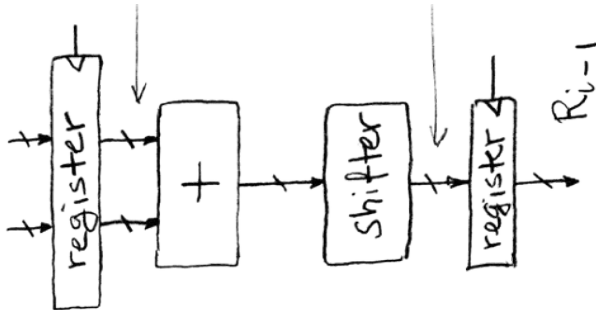
- Register clk-to-q: 30 ps
- Register setup: 20 ps
- Adder delay: 150 ps
- Shifter delay: 50 ps

Critical path delay = $t_{clk_q} + t_{Add} + t_{Setup}$

$$= 30 + 150 + 20 \text{ ps} = 200 \text{ ps}$$

Latency = 2 cycles = 400 ps

Throughput = 1/cycle = 5 GHz



Latency, Throughput

Single Cycle:

Latency = 250 ps

Throughput = $1/\text{cycle} = 4 \text{ GHz}$

Pipeline:

Latency = 400 ps

Throughput = $1/\text{cycle} = 5 \text{ GHz} = 1.25\times$

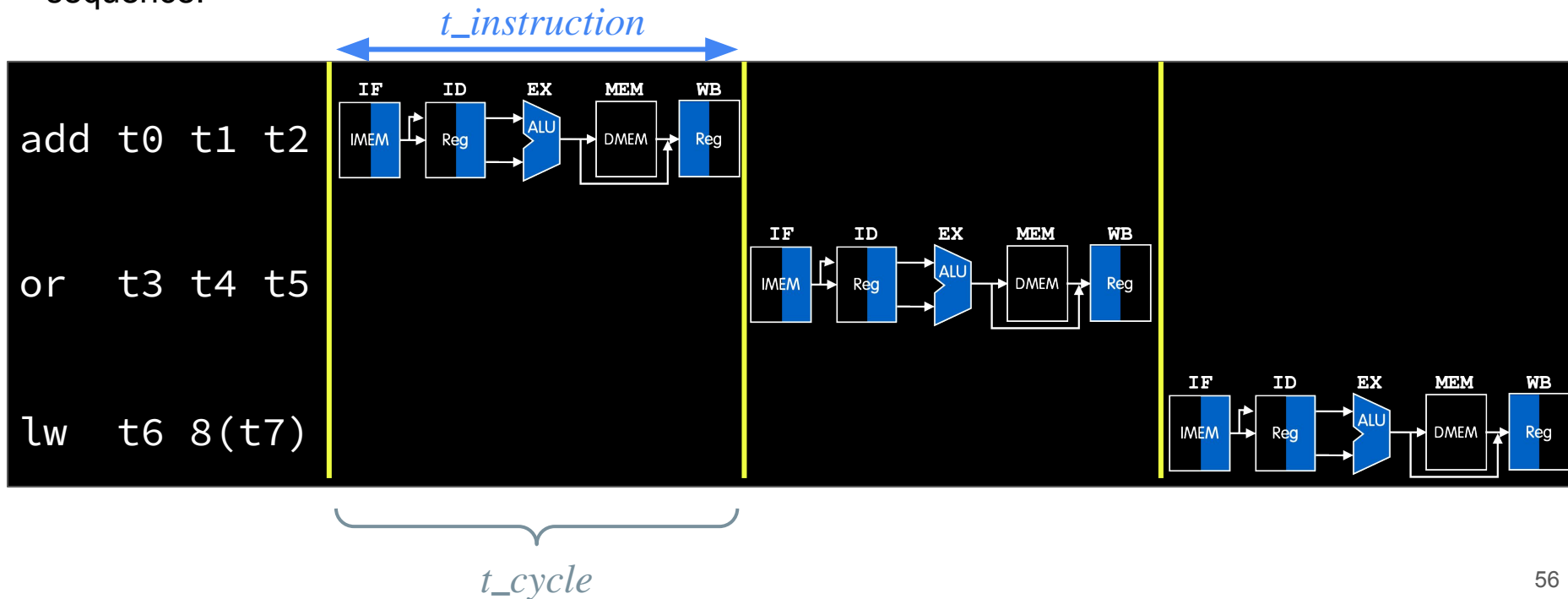
- When we pipeline, latency can increase.
- Two stage pipeline can up to double throughput
- Why can't we get exactly 2x throughput?
 - Unequal split between pipeline stages
 - Register setup/clk-to-q happen twice in a single instruction

Agenda

- Instruction Timing
- Performance Measures
- "Iron Law" of Processor Performance
- Intro to Pipelining: Laundry
- Aside: Energy Efficiency
- RISC-V: Improving Throughput
- **RISC-V Pipelined Datapath**

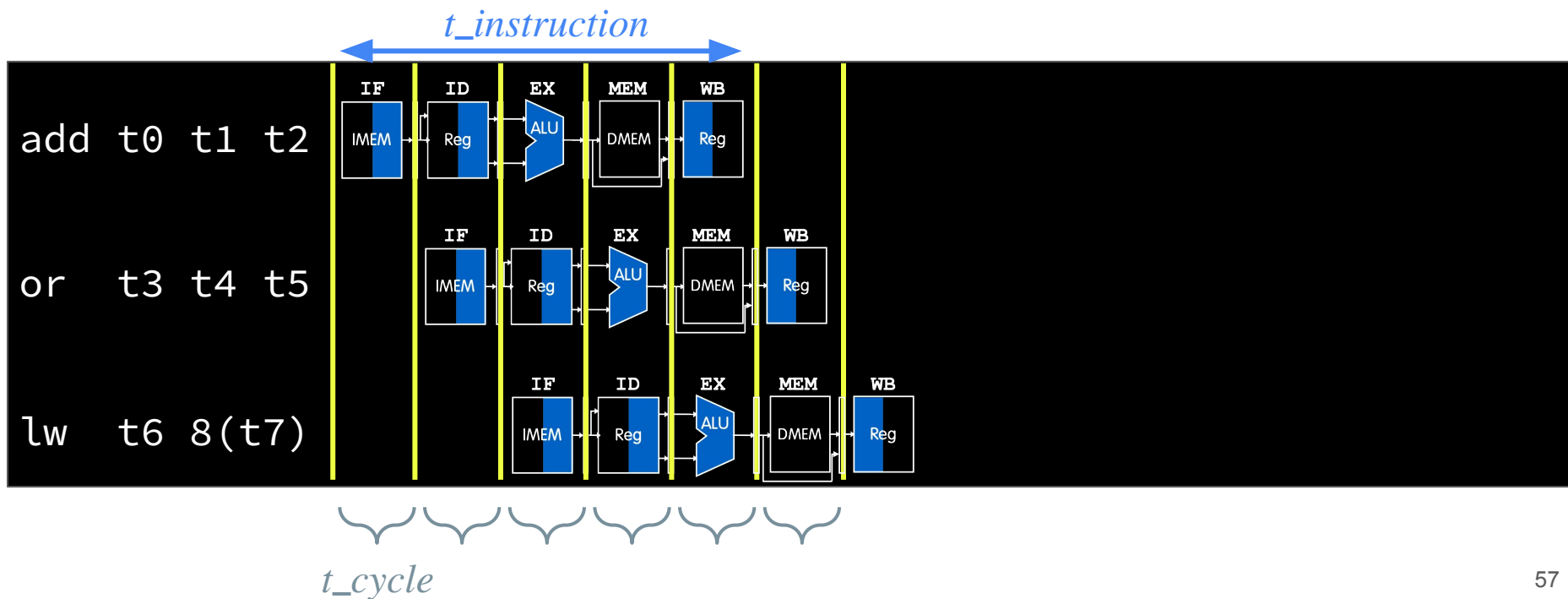
“Sequential” RISC-V Datapath

In a **single-cycle CPU**, only one instruction can access any resources in one clock cycle, in sequence.



Pipelined RISC-V Datapath

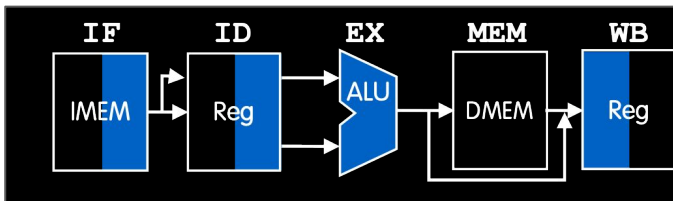
In a **pipelined CPU**, multiple instructions access resources in one clock cycle.



Performance Comparison

Single-Cycle Datapath

High-level
diagram



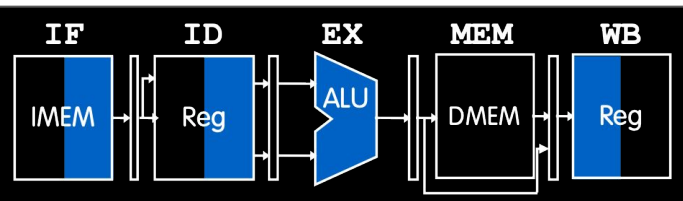
Stage Timing

$t_{stage} = 200, 100, 200, 200, 100 \text{ ps}$
(Reg stages ID, WB only 100 ps)

Instruction
Latency

$t_{instruction} = 800 \text{ ps}$

Pipelined Datapath



$t_{stage} = 200 \text{ ps}$
All stages same length

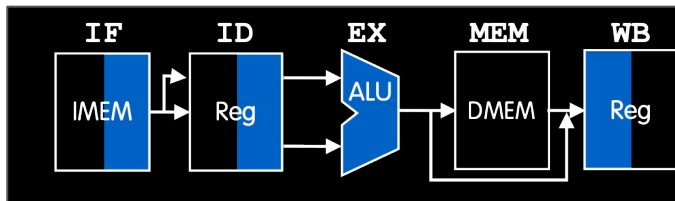
$t_{instruction} = 5 t_{cycle} = 1 \text{ ns}$

- The pipelined CPU still uses one clock for all stages!
- Clock cycle time is limited by the slower stages.

Performance Comparison

Single-Cycle Datapath

High-level
diagram



Stage Timing

$t_{stage} = 200, 100, 200, 200, 100 \text{ ps}$
(Reg stages ID, WB only 100 ps)

Instruction
Latency

$t_{instruction} = 800 \text{ ps}$

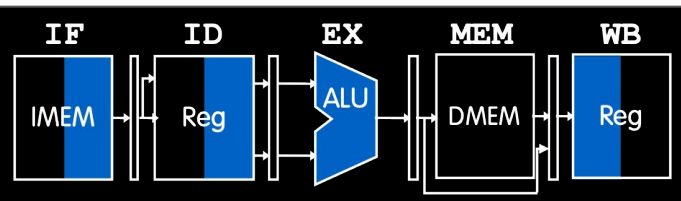
Clock
frequency

$1/t_{instruction} = 1/800 \text{ ps}$
 $= 1.25 \text{ GHz}$

CPI

~ 1

Pipelined Datapath



$t_{stage} = 200 \text{ ps}$
All stages same length

$t_{instruction} = 5 t_{cycle} = 1 \text{ ns}$

$1/t_{cycle} = 1/200 \text{ ps}$
 $= 5 \text{ GHz (4x speedup)}$
 ~ 1

(<1 actual; see CS152)

Summary

- By pipelining our CPU, we can achieve much higher throughput, by decreasing our critical path, and thus increasing our clock frequency
- Sadly, this doesn't work quite yet.
 - Unlike laundry loads, instructions rely on each other. Running instructions in parallel naively leads to wrong answers.
- Tomorrow: How to resolve these issues and get a pipelined CPU that works.